```java
class TicketBooking {
    private String Name;
    private String status;

    public TicketBooking(String Name) {
        if (Name == null || Name.trim().isEmpty()) {
            this.Name = "Unknown Passenger";
            this.status = "Failed";
        } else {
            this.Name = Name;
            this.status = "Booked";
        }
    }

    public void displayDetails(String threadInfo) {
        System.out.println("Passenger Name : " + Name);
        System.out.println("Booking Status : " + status);
        System.out.println("Thread Info    : " + threadInfo);
    }

    public String getName() {
        return Name;
    }
}

class BookingThread extends Thread {
    private TicketBooking booking;

    public BookingThread(TicketBooking booking, String threadName) {
        super(threadName);
        this.booking = booking;
    }

    @Override
    public void run() {
        try {
            if (booking != null) {
                Thread.sleep(500);
                String threadInfo = getName() + " (ID: " + getId() + ")";
                booking.displayDetails(threadInfo);
            } else {
                System.out.println(getName() + ": Error - Booking object is null.");
            }
        } catch (InterruptedException e) {
            System.out.println(getName() + " was interrupted during booking.");
        }
    }
}

class BookingRunnable implements Runnable {
    private TicketBooking booking;

    public BookingRunnable(TicketBooking booking) {
        this.booking = booking;
    }

    @Override
    public void run() {
        try {
            if (booking != null) {
                Thread.sleep(500);
                Thread current = Thread.currentThread();
                String threadInfo = current.getName() + " (ID: " + current.getId() + ")";
                booking.displayDetails(threadInfo);
            } else {
                System.out.println(Thread.currentThread().getName() + ": Error - Booking object is null.");
            }
        } catch (InterruptedException e) {
            System.out.println(Thread.currentThread().getName() + " was interrupted during booking.");
        }
    }
}

public class multithre {
    public static void main(String[] args) {
        System.out.println("=== Railway Reservation Multithreading System ===\n");

        TicketBooking booking1 = new TicketBooking("Rahul");
        TicketBooking booking2 = new TicketBooking("Priya");
        TicketBooking booking3 = new TicketBooking("Aman");
        TicketBooking booking4 = new TicketBooking("Neha");

        BookingThread thread1 = new BookingThread(booking1, "Thread-Rahul");

        Thread thread2 = new Thread(new BookingRunnable(booking2), "Thread-Priya");
        Thread thread3 = new Thread(new BookingRunnable(booking3), "Thread-Aman");
        Thread thread4 = new Thread(new BookingRunnable(booking4), "Thread-Neha");

```

```
        System.out.println("Starting all booking threads concurrently...\n");
        thread1.start();
        thread2.start();
        thread3.start();
        thread4.start();

        try {
            thread1.join();
            thread2.join();
            thread3.join();
            thread4.join();
        } catch (InterruptedException e) {
            System.out.println("Main thread interrupted while waiting for completion.");
        }

        System.out.println("All passenger booking requests have been processed successfully!");
    }
}
```

```
#include <iostream>
using namespace std;

int binarySearch(int A[], int low, int high, int target) {
    if (low > high)
        return -1;
    int mid = (low + high) / 2;
    if (A[mid] == target)
        return mid;
    else if (target < A[mid])
        return binarySearch(A, low, mid - 1, target);
    else
        return binarySearch(A, mid + 1, high, target);
}

int main() {
    int n;
    cout << "Enter the number of elements: ";
    cin >> n;
    int A[n];

    cout << "Enter " << n << " elements in Ascending order:\n";
    for (int i = 0; i < n; i++) {
        cin >> A[i];
    }
    int target;
    cout << "Enter the element to be searched: ";
    cin >> target;

    int pos = binarySearch(A, 0, n - 1, target);
    if (pos != -1)
        cout << "Element found at position " << pos + 1;
    else
        cout << "Element not found";

    return 0;
}

```

```cpp
#include <iostream>
using namespace std;

int main(){
    int n;
    cout<<"Enter size of the array: ";
    cin>>n;
    int a[n];
    cout<<"Enter the elements of the array: ";
    for(int i=0;i<5;i++){
        cin>>a[i];
    }
    cout<<"Enter the element to be found: ";
    int x;
    cin>>x;
    bool found = false;
    int index = -1;
    for(int i=0;i<5;i++){
        if(a[i]==x){
            index = i;
            found = true;
            break;
        }
    }
    if(found){
        cout<<"Element found at index "<<index<<endl;
    }else{
        cout<<"Element not found"<<endl;
    }
}
```